ProjNaN

AI-Assisted Collaborative Project Tracking and Progress Management System

Project Proposal for UCS503P
Submitted to: Dr. Anushka
Thapar Institute of Engineering and Technology

Vanshaj Sharma, CSED* Aryan Thakkar, CSED†

September 16, 2026

1 Higher-order goal

. . . secondary framing

This project aims to improve the transparency, accountability, and efficiency of collaborative software projects in educational environments by providing a unified platform for planning, tracking, and evaluating team progress. While the immediate target is a classroom containing multiple student project teams, the system is designed as a general project-monitoring platform that can later be extended to university-level research projects, student organizations, and organizational teams.

The immediate engineering objective is to translate project planning and repository activity into a measurable, centralized, and deployable software workflow through task management, GitHub integration, automated progress analysis, and role-based dashboards.

2 Time-to-value

. . . primary heuristic

- We will deliver meaningful increments quickly by first implementing the core project-management workflow: class creation, team formation, task creation, task tracking, and teacher dashboards.
- GitHub integration will then be introduced to automatically collect repository activity such as commits, pull requests, and issues.
- AI-assisted task verification will be added as an incremental layer that compares repository activity with predefined project tasks and identifies potentially completed tasks.
- Weekly development iterations will be used to validate individual workflows early and reduce integration risk.
- Success will be defined by how effectively the first usable version allows a teacher to understand project progress without manually inspecting every team's repository.

*Roll No: 1024160114, Email: vsharma2_be24thapar.edu
†Roll No: 1024160100, Email: athakkar_be24thapar.edu

3 Problem Statement

In project-based courses, instructors often supervise multiple teams simultaneously. A class may contain several teams of two to three students, with each team working on a different software project. Monitoring the actual progress of these projects can become difficult because planning, task tracking, source-code activity, and communication are frequently distributed across different platforms.

Common pain points include:

- **Fragmentation:** tasks may be maintained in notebooks, messaging applications, spreadsheets, or issue trackers while the actual implementation exists separately in GitHub repositories.
- **Limited instructor visibility:** teachers may have to manually inspect repositories, commits, pull requests, or demonstrations to determine whether a team is making meaningful progress.
- **Weak individual accountability:** a project can appear active even when the workload is unevenly distributed among team members.
- **Manual progress updates:** students must repeatedly update task status even when evidence of their work already exists in repository activity.
- **Difficulty identifying stalled teams:** an instructor may not immediately know which teams have stopped progressing or which planned milestones are falling behind.
- **Lack of structured feedback:** teachers have limited centralized information with which to provide timely and targeted suggestions.
- **Low motivation for consistent progress:** traditional project tracking generally provides little incentive for students to complete smaller tasks consistently.

Why this matters now (contextual relevance): As software projects increasingly rely on Git-based development workflows, repository activity provides a valuable source of project-progress information. However, GitHub activity alone does not directly indicate whether predefined project objectives or tasks have been completed. A system that connects planned tasks with actual development activity can reduce manual monitoring while improving transparency and accountability.

4 Proposed Solution *. . . Software Proposition*

4.1 Overview

Build a **web-based "ProjNaN" system** that provides teachers and student teams with a centralized project-tracking workspace with the following components:

- **Class management:** teachers create a class and receive a unique class code that students can use to join.
- **Team management:** students are organized into project teams, with each team having its own project workspace.
- **Project planning:** teams define project goals, milestones, and To-Dos with daily, weekly, or monthly deadlines.

- **GitHub integration:** each project can be linked to a GitHub repository so that relevant development activity can be retrieved automatically.
- **AI-assisted task verification:** repository activity is analyzed against pending To-Dos to identify activities that potentially satisfy specific project tasks.
- **Progress dashboards:** teachers can view team-level and class-level progress from a centralized dashboard.
- **Individual contribution tracking:** students can view their own goals, completed tasks, and contribution activity.
- **Gamification:** contribution points, achievements, and a class leaderboard can encourage consistent participation.
- **Feedback and recommendations:** teachers can provide feedback and recommendations to teams based on their progress.
- **Progress alerts:** the system can highlight teams or tasks that are overdue, inactive, or significantly behind their planned milestones.

4.2 Core workflow

1. A teacher creates a class and the system generates a unique class code.
2. Students join the class using the code and form or are assigned to project teams.
3. Each team creates a project workspace and defines project goals, milestones, and To-Dos.
4. To-Dos can be assigned to specific team members and given daily, weekly, or monthly deadlines.
5. The team links its GitHub repository with the project workspace.
6. The backend periodically retrieves relevant GitHub activity such as commits, pull requests, and issues.
7. The system maps repository activity to existing project tasks using task metadata and AI-assisted semantic matching.
8. Potentially completed tasks are identified and assigned a confidence score.
9. Confirmed task completion updates the team's project-progress indicators.
10. Students can inspect their individual contribution and progress.
11. Teachers can view the progress of all teams through a centralized dashboard.
12. Based on project status, teachers can provide feedback or assign additional tasks.
13. Contribution points can be updated as students complete meaningful tasks, enabling a class leaderboard.

4.3 User roles

The system will use role-based access control:

- **Teacher/Admin:** creates classes, manages teams, monitors projects, views analytics, provides feedback, and manages tasks.
- **Team member:** creates and updates project tasks, connects GitHub, monitors team progress, and views contributions.
- **Student:** manages assigned individual goals and views personal contribution statistics.
- **System/AI service:** processes repository activity and generates task-alignment suggestions.

4.4 Operational constraints for an educational setting

- Keep the solution usable on standard student laptops and mobile-responsive interfaces.
- The core project-tracking functionality must remain usable even if GitHub integration or AI analysis is temporarily unavailable.
- AI should assist progress verification rather than becoming a single point of failure for task management.
- Repository access must follow GitHub authorization and permissions rather than requiring students to expose private repositories unnecessarily.
- The initial implementation should prioritize workflow correctness, reliability, and usability over sophisticated AI research.
- The system should be deployable using commonly available web hosting, managed databases, and API-based services.

5 Solution Approach *. . . Engineering focus*

This is an engineering course project, so the primary focus will be on building a reliable project-management workflow and integrating existing software services. AI will be used as an intelligent assistance layer rather than as the primary research contribution.

5.1 Project and task management *. . . workflow design*

Each project will contain project objectives, milestones, tasks/To-Dos, task priority, task owner, task type or frequency, deadline, status, completion evidence, and related GitHub activity.

Tasks may follow states such as:

Planned → In Progress → Verification Pending → Completed

This provides a consistent state machine for tracking project progress.

5.2 GitHub integration *. . . repository activity*

The system will integrate with GitHub through its available APIs and, where appropriate, webhooks. Relevant repository events may include commits, pull requests, issues, pull-request merges, and branch activity.

Repository events will be stored as project activity records. The system will associate these events with students and teams where possible. The integration provides objective development evidence without requiring students to manually report every development action.

5.3 AI-assisted task verification *... intelligent assistance*

The AI subsystem will compare a project's predefined To-Dos with relevant repository activity.

For example, a task such as "Implement JWT-based user authentication" may be compared against repository activity containing commits or pull requests describing authentication, token handling, login middleware, or related implementation changes.

The system will generate a task-alignment assessment containing:

- Candidate task.
- Relevant repository activity.
- Alignment/confidence score.
- Short explanation of the detected relationship.
- Verification state.

Rather than automatically assuming that every matching commit proves completion, the system will treat AI output as **evidence for task verification**. This reduces the risk of incorrectly marking tasks as complete because of vague or misleading commit messages.

5.4 Gamification *... contribution incentives*

The system will introduce lightweight gamification to encourage consistent contribution. Possible mechanisms include points for verified task completion, milestone-completion bonuses, contribution streaks, team achievements, and a class leaderboard.

Points will primarily reward meaningful, verified project activity rather than raw commit count. This avoids encouraging students to generate many small or meaningless commits merely to increase their score.

5.5 Web architecture *... web-based solution*

A typical three-tier architecture will be used:

- **Frontend:** responsive web interface containing student workspaces, teacher dashboards, task boards, project analytics, and leaderboard views.
- **Backend API:** authentication, authorization, project management, task state transitions, GitHub integration, AI analysis, scoring, and notification logic.
- **Database:** stores users, classes, teams, projects, tasks, milestones, repository metadata, GitHub activity, AI verification results, feedback, and contribution records.

The system will maintain clear separation between user-facing workflows, business logic, external integrations, and persistent data.

5.6 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build the application on every change.
- Run unit and integration tests.
- Perform linting and basic static checks.

- Validate backend API functionality.
- Produce repeatable deployment artifacts.
- Deploy verified changes to a staging environment.
- Enable frequent incremental releases.

6 Evaluation Criterion *... measurable and attributable*

We define success using metrics connected directly to the project's primary goal: improving project-progress visibility and reducing manual monitoring effort.

6.1 Primary evaluation metric

Progress Monitoring Time (PMT): the time required for an instructor to determine the current progress state of all teams in a class.

- **Target:** reduce the time required to obtain a reliable class-level progress overview compared with manually inspecting project repositories and student updates.
- **Attribution:** measured using task-completion records and timed pilot evaluations with and without the system.

6.2 Secondary evaluation metrics

- **Task verification accuracy:** percentage of AI-generated task-alignment suggestions that are correctly classified during evaluation.
- **Progress visibility:** percentage of active projects for which the instructor can identify current milestone/task status from the dashboard.
- **Manual update reduction:** reduction in the number of project-status updates that students must enter manually when relevant GitHub evidence is available.
- **Individual contribution visibility:** ability to identify individual task ownership and verified contribution within a team.
- **User satisfaction:** student and instructor feedback collected using a simple 1–5 usability/clarity questionnaire.
- **System reliability:** availability of core project-management functionality during the pilot.
- **Detection of inactive projects:** ability to identify teams with overdue tasks or prolonged periods of low repository activity.

6.3 Pilot validation plan *... high-level*

- Conduct a classroom-scale simulation using multiple project teams.
- Include one instructor/admin account and approximately 3–10 simulated or real project teams with two to three students per team.
- Create multiple tasks with different deadlines and priorities and link representative GitHub repositories.

- Compare the instructor's ability to determine project status manually against the information available through ProjNaN.
- Evaluate AI task-verification results against manually labeled examples to estimate the usefulness and accuracy of the matching system.

7 Scalability *... with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in users, teams, projects, tasks, and repository activity by:
   - decoupling frontend and backend components,
   - enabling pagination and indexing for common queries,
   - using stateless API design where appropriate,
   - processing GitHub synchronization and AI analysis asynchronously.
2. **Operationally deployable (practically):** The system should run on common web hosting using:
   - managed database infrastructure,
   - containerized or platform-hosted deployment,
   - CI/CD pipeline for staging and production.

The initial class-level implementation can later be extended to multiple classes, university-wide project management, professor-managed research groups, departments, and organizational teams without fundamentally changing the core project-task model.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a web-based project:

- Modern web framework for the frontend.
- Backend framework for REST APIs and business logic.
- Managed relational database for structured project data.
- Authentication and role-based authorization.
- GitHub API/webhook integration for repository activity.
- LLM/API service for semantic task-activity matching.
- Background job mechanism for asynchronous synchronization and AI processing.
- Test framework for unit and integration tests.
- CI runner for automated testing and builds.
- Cloud or platform-based staging deployment.

We will use these mature components to focus on workflow design, correctness, integration, testing, observability, and measurement rather than implementing infrastructure or machine-learning models from scratch.

9 Project scope and deliverables *. . . iteration-friendly*

9.1 Initial deliverable *. . . first iteration*

- Teacher/admin authentication.
- Class creation and unique class-code generation.
- Student joining through class code.
- Team creation and team-member management.
- Project creation.
- Task/To-Do creation, assignment, and status management.
- Project and team progress dashboard.
- Basic individual contribution tracking.
- GitHub repository linking.
- Basic GitHub activity retrieval.
- CI pipeline with automated tests and linting.
- Deployment to a staging environment.

9.2 Subsequent deliverables

- Automated GitHub activity synchronization.
- AI-assisted task/activity matching.
- Confidence-based task verification with supporting evidence.
- Milestone and deadline tracking.
- Teacher feedback and recommendations.
- Individual contribution dashboard.
- Contribution points and achievements.
- Class leaderboard.
- Notifications for overdue tasks and project inactivity.
- Improved project analytics and filtering.
- Background processing for large repository activity.
- Improved scalability and production deployment.

10 Risks and mitigations

- **Incorrect AI task verification:** AI may incorrectly associate repository activity with a task. Mitigation: use confidence scores, retain evidence, and allow human/team verification instead of blindly marking tasks complete.
- **Gaming of contribution points:** students may generate meaningless commits to increase their score. Mitigation: avoid scoring raw commit counts and prioritize verified task completion, pull requests, and meaningful project milestones.
- **GitHub API limitations:** repository synchronization may be affected by API limits or authorization issues. Mitigation: cache activity, synchronize incrementally, and ensure the core task-management system works independently of GitHub.
- **Privacy and repository access:** projects may contain private or sensitive code. Mitigation: use authorized GitHub access, minimize stored repository data, and follow least-privilege principles.
- **AI service availability/cost:** external AI services may become unavailable or expensive. Mitigation: isolate AI functionality behind a service layer and retain a manual task-verification workflow.
- **Uneven team participation:** students may have different responsibilities that are not reflected by repository activity alone. Mitigation: combine task ownership, verified activity, and manually recorded contributions rather than relying on a single metric.
- **Evaluation ambiguity:** project progress can be subjective. Mitigation: define measurable task states, milestones, timestamps, and verification criteria before evaluation.
- **Scope expansion:** supporting university-wide or company-wide workflows too early may increase complexity. Mitigation: limit the initial implementation to a single class while designing extensible entities such as organizations, classes, teams, and projects.

11 Summary

This project proposes **ProjNaN**, a deployable web-based platform for monitoring and managing collaborative student software projects. The system connects structured project planning with actual development activity by integrating task management, GitHub repository activity, AI-assisted task verification, progress dashboards, and lightweight gamification.

The initial implementation will focus on a classroom containing multiple teams, allowing an instructor to create a class, organize teams, monitor project tasks, connect repositories, and obtain a centralized view of progress. Students will be able to manage individual and team goals while receiving visibility into their contributions.

The project meets the software-engineering objectives by emphasizing a structured multi-user workflow, role-based access, database-backed state management, third-party API integration, automated testing, CI/CD, measurable evaluation, and scalability. AI is used as an assisting component for interpreting repository activity rather than being treated as the sole mechanism for project management.

The architecture is intentionally extensible so that the same project-team-task model can later support university research projects, departments, and organizational teams. The immediate goal, however, remains a reliable and measurable classroom project-tracking system that reduces the manual effort required to understand project progress while improving accountability and transparency.